

UML

Trentin Tommaso

December 27, 2025

Contents

1	Introduzione	2
1.1	Motivazione	2
1.2	Terminologia	2
1.3	Linguaggi di modellazione	2
1.3.1	UML: Idea	2
1.4	Regole Prescrittive / Descrittive	2
1.5	UML: Utilizzi	2
1.5.1	Bozze: Espressività & Completezza	2
1.5.2	Progetto Dettagliato: Espressività + Completezza	2
1.5.3	Linguaggio di Programmazione	2
1.6	Informazioni Soppresse	2
1.7	Tipi di Diagrammi	3
1.8	Integrazione nel Processo di Sviluppo	3

1 Introduzione

1.1 Motivazione

Moderni sistemi SW tendono a crescere e diventare sempre più complessi → impensabile comprenderli direttamente dal codice. Nasce quindi necessità di forme di rappresentazione astratte → nascita **modellazione**.

1.2 Terminologia

- **Modello**: astrazione che descrive sistema / sottosistema;
- **Vista**: descrizione degli aspetti fisici di un sistema da una certa prospettiva (si omettono dettagli non rilevanti);
- **Notazione**: insieme di elementi grafici / testuali e regole per rappresentare le viste.

1.3 Linguaggi di modellazione

In passato erano utilizzati diversi linguaggi di modellazione, negli ultimi anni si sta affermando UML (Unified Modelling Language) come linguaggio unificato per essere utilizzato in tutte le attività di modellazione.

1.3.1 UML: Idea

Famiglia di notazioni grafiche che si basano su un singolo **meta-modello** (modello che definisce concetti stessi del linguaggio di modellazione, indica secondo quali regole sia possibile costruire modelli UML). UML *non è una metodologia*: il suo obiettivo è fornire un supporto al processo di sviluppo SW.

1.4 Regole Prescrittive / Descrittive

- Regole **prescrittive**: stabilite da organismi standardizzanti che definiscono precisamente lessico, sintassi e semantica del linguaggio;
- Regole **descrittive**: stabilite per convenzione comune, possono essere meglio comprese guardando come UML viene usato nella pratica da un'organizzazione;

Bisogna conoscere sempre convenzioni particolari della specifica organizzazione e del singolo progetto, anche se al di fuori dello standard.

1.5 UML: Utilizzi

- **Bozza**: tracciare modello informale di un sistema da realizzare (*forward engineering*) o per descrivere sistemi esistenti (*reverse engineering*);
- **Progetto dettagliato (blueprint)**: realizzare un modello completo della soluzione architeturale del sistema;
- **Linguaggio di programmazione**: in grado di modellare completamente e precisamente il sistema SW.

1.5.1 Bozze: Espressività & Completezza

Scopo: aiutare comunicazione e discussione idee, esplorando soluzioni alternative, diagrammi non devono essere esaustivi / definire tutti gli aspetti del codice. Vengono spesso disegnate collaborativamente in poco tempo, non rispettando tutte le regole formali dello standard.

1.5.2 Progetto Dettagliato: Espressività + Completezza

Scopo: aiutare comprensione e completezza. Progettista sviluppa modello di progetto che sviluppatore dovrà realizzare (salvandolo in file condivisi). Completezza e non ambiguità aiutano il programmatore che verrà guidato dal modello e non dovrà interpretarne aspetti ambigui.

1.5.3 Linguaggio di Programmazione

Approccio **MDA** (Model Driven Architecture) esplora la possibilità di usare UML come linguaggio di programmazione, volendo stabilire una sintassi e una semantica per UML in modo da portare a generazione automatica di codice eseguibile rappresentativo del modello. Sfida: modellare precisamente anche la logica del progetto, Vantaggio: generazione codice per diverse piattaforme target partendo da un modello indipendente dalla piattaforma.

1.6 Informazioni Soppresse

Assenza di qualche informazione da un diagramma UML non implica, in generale, che tale informazione non esista (alcuni aspetti potrebbero essere assenti perché non ancora trattati al momento della stesura del diagramma).

1.7 Tipi di Diagrammi

UML 2 possiede 13 tipi differenti di diagrammi ufficiali, appartenenti a due categorie:

- **Diagrammi strutturali:** modellano organizzazione del sistema;
- **Diagrammi comportamentali:** modellano comportamento e interazioni tra le entità del sistema.

Tali diagrammi rappresentano i deliverables di diverse fasi del ciclo di vita del SW, tra cui analisi di requisiti e attività di progettazione di alto / basso livello.

Diagramma	Capitoli del libro	Scopo	Origine
Attività	11	comportamento procedurale e parallelo	UML 1
Classi	3, 5	classi, loro caratteristiche e relazioni	UML 1
Comunicazione	12	interazione tra oggetti con enfasi sui collegamenti	chiamato "diagramma di collaborazione" in UML 1
Componenti	14	struttura dei componenti e loro connessioni	UML 1
Struttura composita	13	scomposizione di una classe a runtime	introdotto in UML 2
Deployment	8	distribuzione di elaborati in diversi nodi	UML 1
Interazione Generale	16	fusione di un diagramma di sequenza con un diagramma delle attività	introdotto in UML 2
Oggetti	6	configurazione esemplificativa di istanze	presente in UML 1 in modo non ufficiale
Package	7	struttura gerarchica al momento della compilazione	presente in UML 1 in modo non ufficiale
Sequenza	4	interazione tra oggetti, con enfasi sulla sequenza	UML 1
Macchina a stati	10	come gli eventi cambiano un oggetto durante il suo ciclo di vita	UML 1
Temporizzazione	17	interazione tra oggetti, con enfasi sul tempo	introdotto in UML 2
Casi d'uso	9	come gli utenti interagiscono con un sistema	UML 1

1.8 Integrazione nel Processo di Sviluppo

- **Analisi dei requisiti:** facilita deduzione dei requisiti, notazione non deve essere troppo complessa per favorire comunicazione con cliente;
- **Progettazione:** modelli più tecnici / dettagliati per descrivere sistema a ingegneri che lo devono implementare;
- **Documentazione:** modelli rendono più semplice la descrizione di parti complesse o convogliano messaggi intuitivamente e immediatamente;
- **Comprensione di SW pre-esistente:** evoluzione / reverse engineering.

Nei processi iterativi ogni iterazione arricchisce i diagrammi delle iterazioni precedenti.